

Sicurezza

CdL in Informatica – L31

Lezione L7/8-th
Programmi e Programmazione

Prof. Emiliano Casalicchio

Department of Computer Science

Sapienza University of Rome



SAPIENZA
UNIVERSITÀ DI ROMA



Learning objective

- Studying security problems in programs and programming, with an eye toward understanding and recognizing various types of software failures.
- Two or more problems may combine in negative ways, some problems may be intermittent or occur only when some other condition is present, and the impact of problems can range from annoying (perhaps not even perceptible) to catastrophic.



Topics

The Terminology of (Lack of) Quality

Programming oversights

- buffer overflows, incomplete mediation, time-of-check to time-of-use errors, off-by-one errors

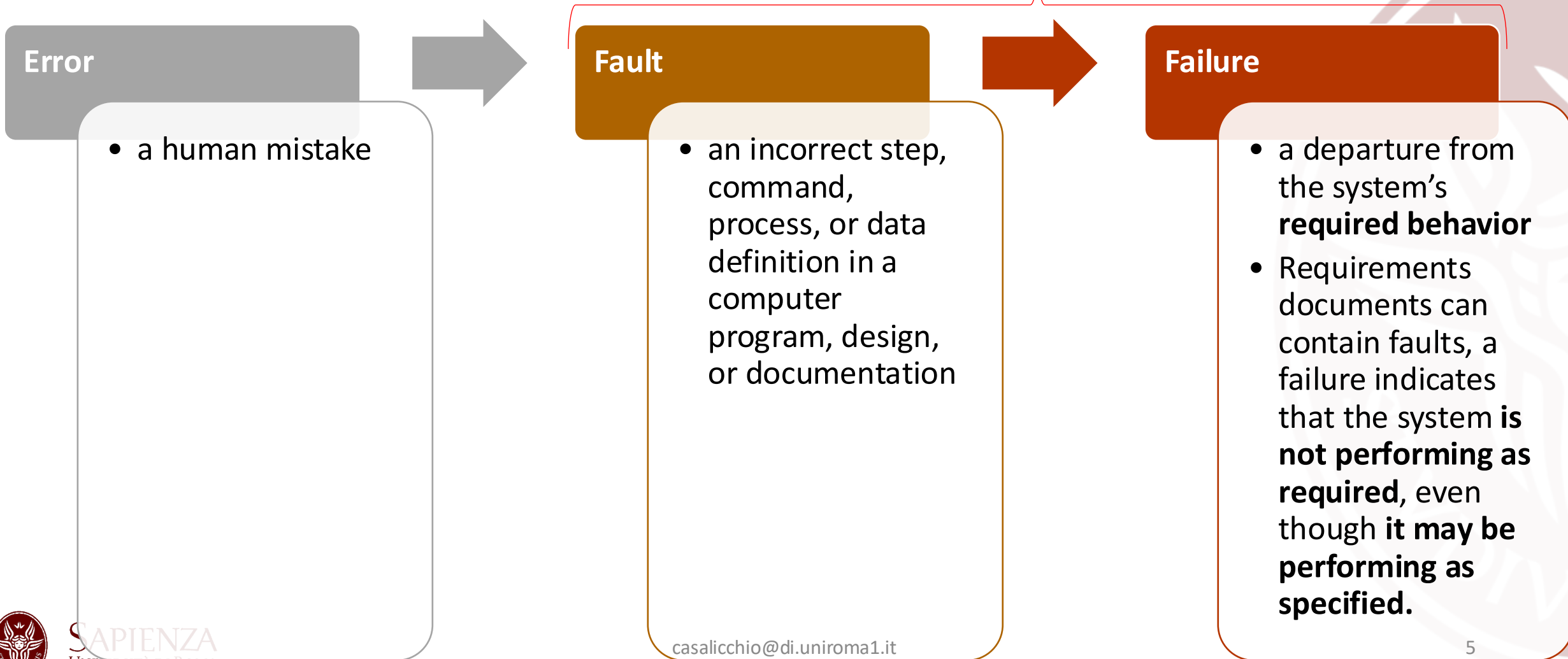
Malicious code

- viruses, worms, Trojan horses
- Developer countermeasures:
 - program development techniques, security principles
- Ineffective countermeasures



The Terminology of (Lack of) Quality

Flow: security engineers made no distinction



Programming oversights

Oversight: an unintentional failure to notice or do something



Unintentional programming oversights

Program flaws can have two kinds of security implications:

- can cause integrity problems leading to harmful output or action
- can offer an opportunity for exploitation by a malicious actor.



Flaw to Integrity violation

- A program flaw can be a fault affecting the correctness of the program's result/operation
- Incorrect operation is an integrity failing
 - Integrity involves not only correctness but also accuracy, precision, and consistency
 - A faulty program can also inappropriately modify previously correct data, sometimes by overwriting or deleting the original data



Flow to Vulnerability

- Even a flaw from a benign cause can be exploited by someone malicious
- If an attacker learns of a flaw and can use it to manipulate the program's behavior, a simple and nonmalicious flaw can become part of a malicious attack



Programming flaws that have security implications

- Buffer Overflow
- Incomplete mediation
- Time-of-Check to Time-of-Use
- Undocumented Access Point (backdoor)
- Off-by-One Error
- Integer Overflow (short)
- Unterminated Null-Terminated String (short)
- Parameter Length, type and Number (short)
- Unsafe Utility program (short)
- Race Condition



Buffer Overflow



Buffer overflow

Buffer overflow **occurs when a program exceeds the space allocated for the buffer**

- locating and preventing such flaws is challenging
- the impact of an overflow can be subtle and disproportionate to the underlying oversight, e.g., forgetting to check for input size
- a buffer overflow is often the first step for mounting a more damaging strike, e.g. opening a shell



Discovery of a buffer overflow: the Dialer.exe example – Litchfield 1999

- Reasoning:
 - Dialerx.exe a utility program to dial number with the modem
 - Dialer.exe was designed to accept phone numbers of different lengths, given country variations, outgoing access codes, and remote signals
 - There should be an upper limit!!
 - Let's try dialer.exe with a 100-digit phone number, the program crashed
- Conclusion
 - The Dialer programmer had probably made an undocumented and untested decision that nobody would ever try to dial a 100-digit phone number
 - **There is a flow ... could it be exploited?**



Dialer.exe, 2nd episode

- Litchfield found that the number to dial was written into the stack
- The dialer.exe program is treated as a program call by the operating system
- **By controlling what dialer.exe overwrote, Litchfield could redirect execution to continue anywhere with any instructions he wanted**
- Buffer overflow is not an old problem:
 - The CVE records database in July 2022 contained 179,663 records, of which buffer overflows accounted for 21,945 entries or over 12%.



Buffer overflow is still here (CVE database)

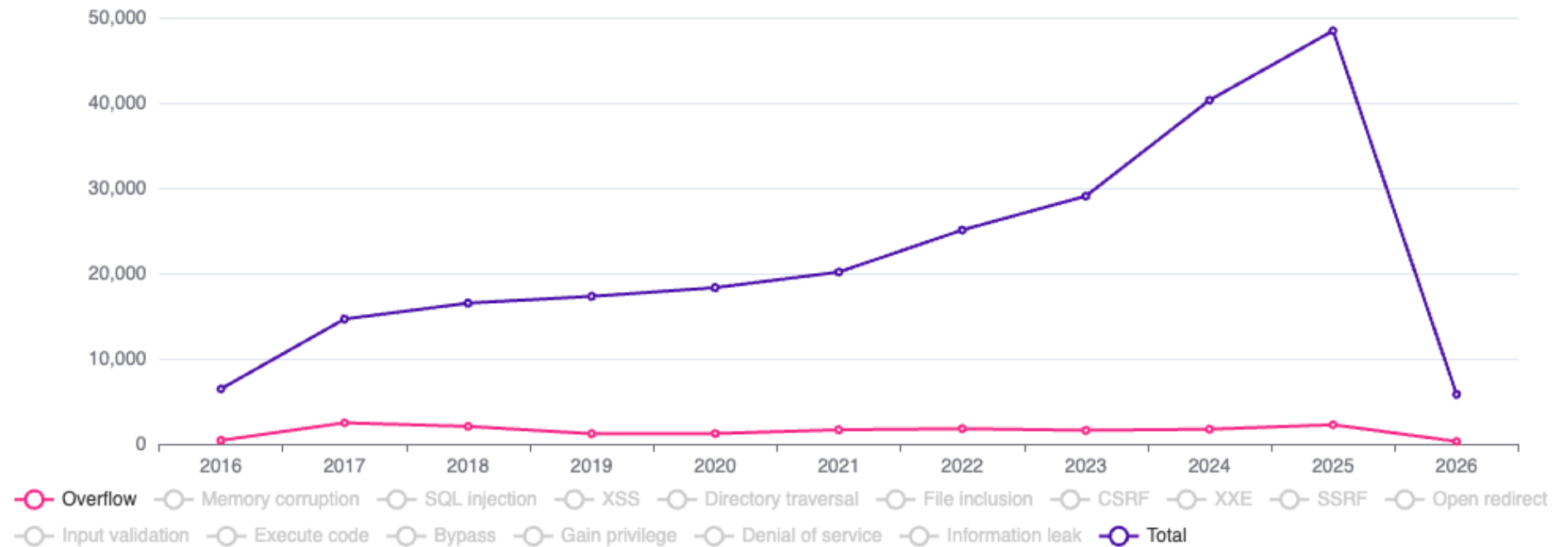
<https://www.cvedetails.com/vulnerabilities-by-types.php>

Year	Overflow
2016	418
2017	2469
2018	2056
2019	1196
2020	1216
2021	1655
2022	1781
2023	1594
2024	1723
2025	2250
2026	296

Total

16654

Vulnerabilities by type & year



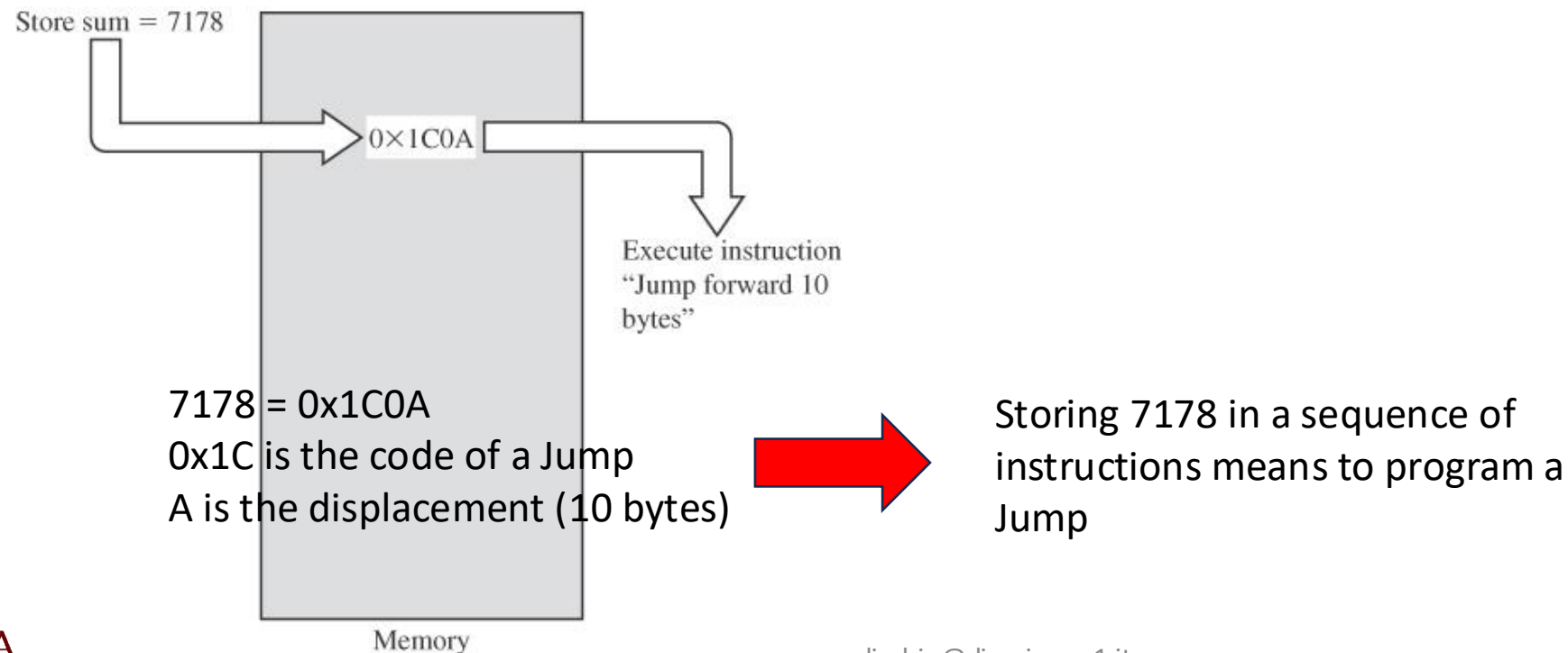
Recall of basic concepts

- The program counter
 - Changing the program counter causes execution to transfer to another instruction stored in the code segment (the usual situation), in the data segment in the heap or in the stack
- Processors support Supervised (protected) and Unsupervised execution mode
 - privileged instructions can be executed only when the processor is running in a protected mode.
 - Trying to execute something that does not correspond to a valid instruction or trying to execute a privileged instruction when not in the proper mode will cause a **program fault**.
 - When hardware generates a program fault, it stops the current thread of execution and transfers control to code that will take recovery action, such as halting the current process and returning control to the supervisor.



Recall of basic concepts

- Code and Data are sequence of bits
 - In memory, code is indistinguishable from data. The origin of code (respected source or attacker) is also not visible



Data as Code

Usually, **programmers** do not treat code as data, or vice versa;

Attackers do it in memory overflow attacks

- The attacker's trick is to cause data to spill over into executable code and then to select the data values such that they are interpreted as valid instructions to perform the attacker's goal
- Buffer overflow attack is a two-step process:
 - first cause the overflow
 - then experiment with data (that is code) injection to cause a desired, predictable result



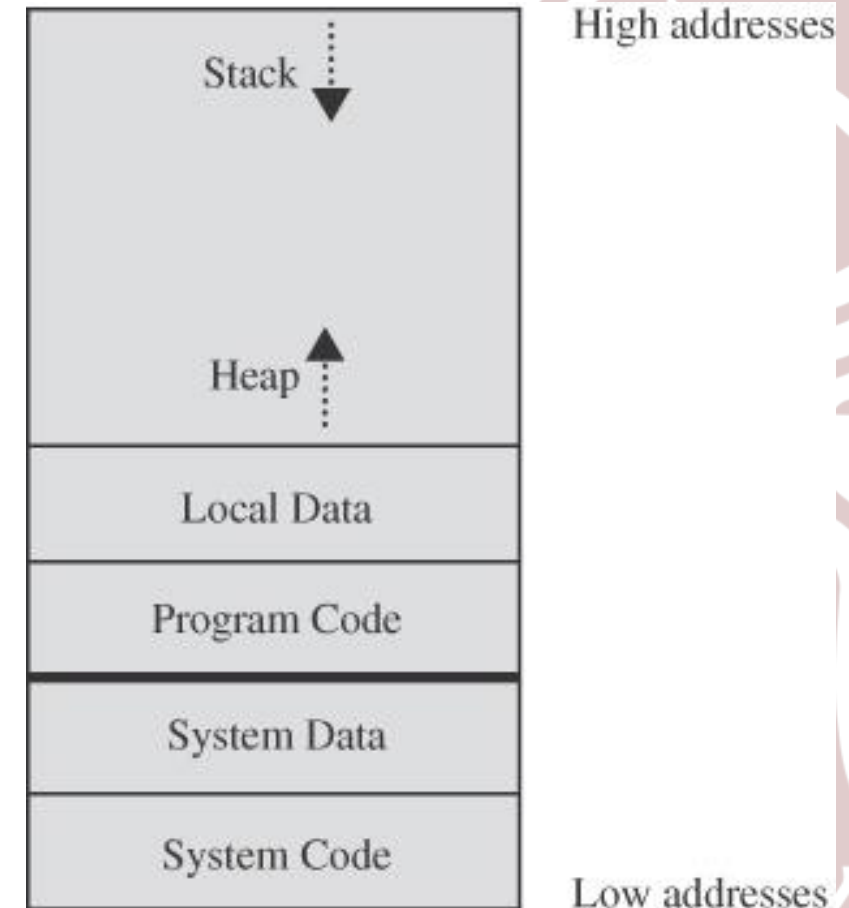
The malicious programmer way of thinking

- What data values could I insert to cause misbehaviour or damage?
- What planned instruction codes could I force the system to execute?



Privilege escalation - replacing the code in the system space

- Every program is invoked by an OS that may run with higher privileges than those of a regular program
- If the attacker can gain control by masquerading as the operating system, the attacker can execute commands with OS privileges
- By replacing a few instructions right after returning from his or her own procedure, the attacker regains control from the operating system, possibly with raised privileges
 - This is the **privilege escalation** technique



Procedure call - the normal behavior

Each time a procedure is called

1. its parameters, the return address, and other local values are pushed onto a stack
2. the old stack pointer is pushed onto the stack
3. the stack pointer register is reloaded with the address of these new values.
4. control is transferred to the subprocedure
5. as the subprocedure executes, it fetches parameters that it finds by using the address pointed to by the stack pointer (stored in a registry)



The attacker action - replacing the code in the stack or heap

By causing an overflow into the stack, the attacker can change

- the old stack pointer
 - changing the context for the calling procedure
- the return address
 - causing control to transfer where the attacker intends when the subprocedure returns

Changing the context or return address allows the attacker to redirect execution to code written by the attacker



Buffer overflow in practice – exercise (at home)

- Pincus, J., and Baker, B. “Beyond Stack Smashing: Recent Advances in Exploiting Buffer Overruns.” *IEEE Security & Privacy*, v2, n4, Jul 2004, p20–27.
- Smashing The Stack For Fun And Profit
 - <https://insecure.org/stf/smashstack.html>
- Mudge's tutorial on writing Buffer overflows
 - https://insecure.org/stf/mudge_buffer_overflow_tutorial.html



Overflow by passing parameters to a web server

- Let assume a routing `userinput.asp` accept `parm1` and the user put a telephone number in the variable `parm1` by means of a web form, e.g.
 - [http://www.somesite.com/subpage/userinput.asp?parm1=\(808\)555-1212](http://www.somesite.com/subpage/userinput.asp?parm1=(808)555-1212)
- Passing a very long string to a web server is a slight variation on the classic buffer overflow, but no less effective.



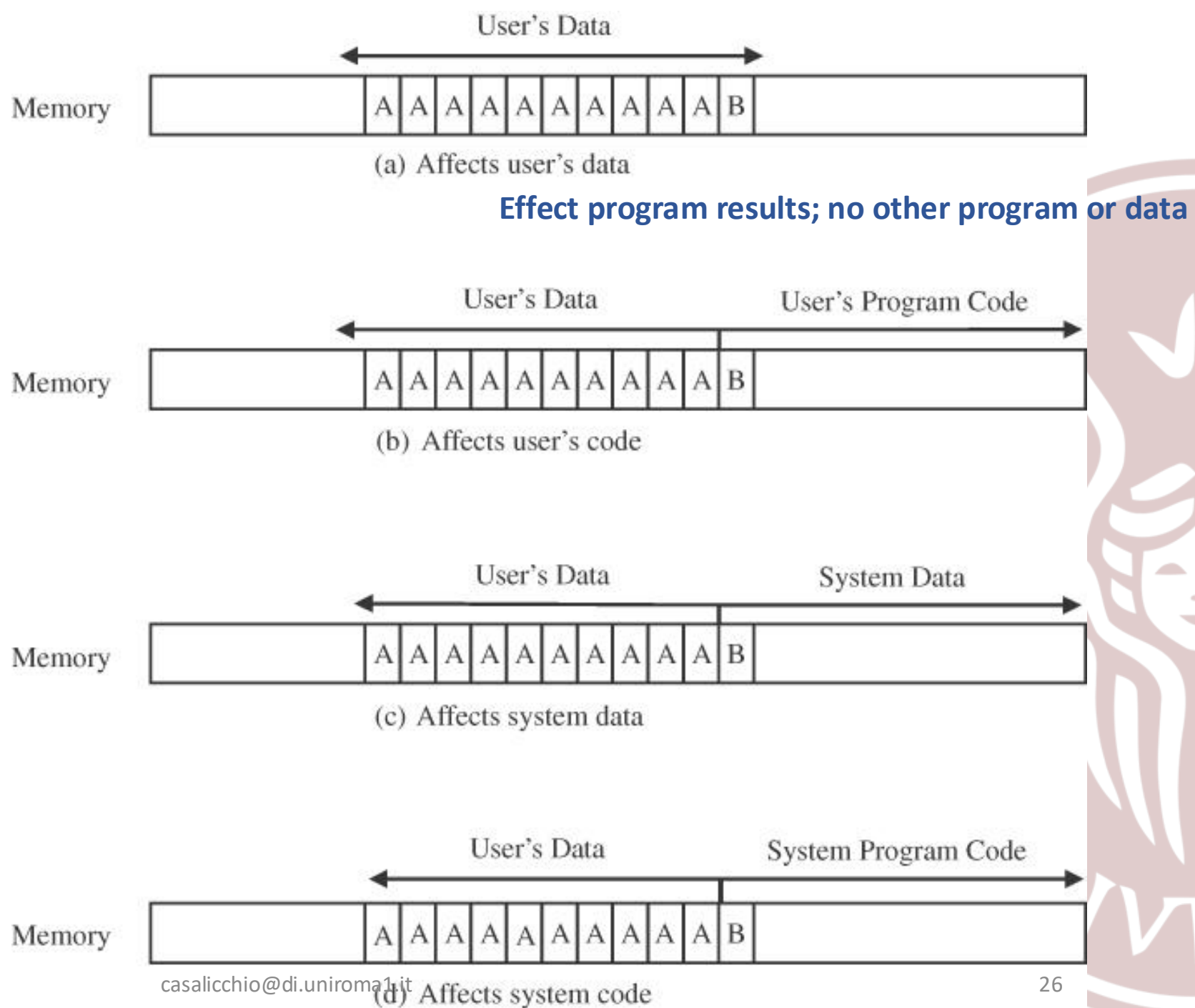
Programming error

Overwriting Memory with Your Own Programs



Programming error: Overwriting Memory with Your Own Programs

```
for (i=0; i<=9; i++)  
    sample[i] = 'A';  
sample[10] = 'B'
```



Overwriting user's space memory

Overwriting user's data segment

- Effect program results; no other program or data

Overwriting user's program code segment

- If it overlays an already executed instruction (which will not be executed again), the user should perceive no effect.
- If it overlays an instruction that is not yet executed, the machine will try to execute an instruction with operation code 0x42
 - If there is no instruction with operation code 0x42, the system will halt on an illegal instruction exception.
 - Otherwise, the machine will use subsequent bytes as if they were the rest of the instruction, with success or failure depending on the meaning of the contents.
- Only the user is likely to experience an effect;
- The fault could appear much later the execution of the overwriting



Overwriting system space memory

Similar effects of overwriting user's data and program code segments, but with **significant security implications: the privilege escalation**

- The attacker overtakes a user's program and now he wants to overtake the operating system, e.g.,
 - to plant permanent code that is reactivated each time a machine is restarted
 - expose passwords or cryptographic keys
- **Attacker's advantage**
 - certain portions of the OS are placed at particular fixed locations, and
 - other data are located at places that can easily be determined during execution



Overwriting system space memory (Cont'd)

The attacker leverage the way OS execute programs

- The OS invokes a user's program as if it were a subprocedure, and the operating system receives control back when the user's program exits
- If the user (that is, an attacker acting as the user) can alter what the operating system does when it regains control, the user can force the operating system to execute code the user wants to run, but with elevated privileges
- Being able to modify operating system code or data allows the user to obtain effective privileged status, accomplishing **privilege escalation**.

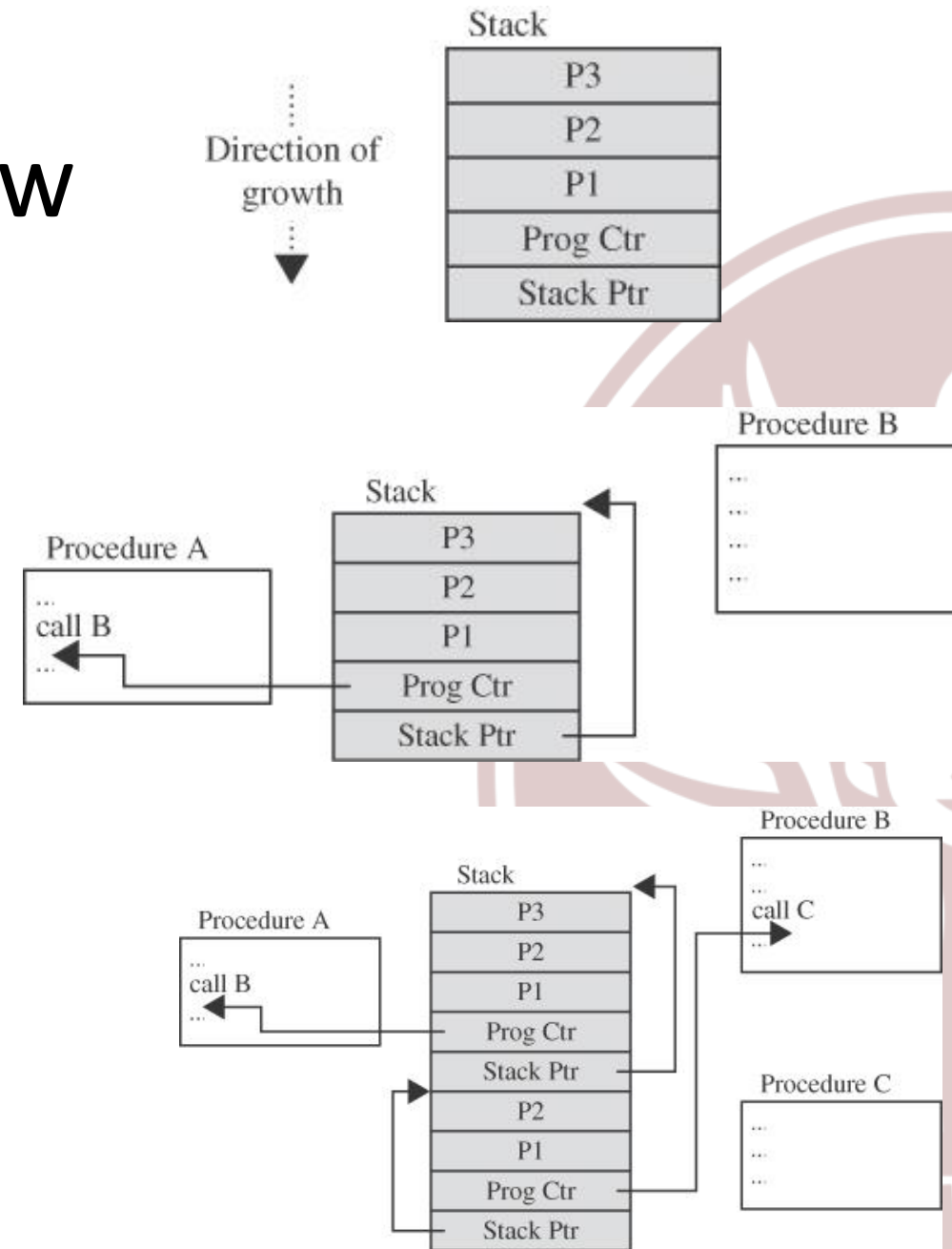
The attacker discovered the possibility for buffer overflow

It can inject data (instructions) that will be executed by the OS when it return to the routine that run the subprogram (your program that contain a flaw)

The overwriting/injection occurs in the stack – We call that **Stack Overflow**

Stack frame & buffer overflow

- **program counter = return address**, the address at which execution should resume when B exits
- Stack pointer, the pointer to the the stack address just before where A pushed values onto the stack
- If the attacker overwrite the program counter, it will redirect program execution after the procedure returns
 - that redirection is a frequently seen step in exploiting a buffer overflow.



```
void hello(char *tag)
{
    char inp[16];

    printf("Enter value for %s: ", tag);
    gets(inp);
    printf("Hello your %s is %s\n", tag, inp);
}
```

(a) Basic stack overflow C code

```
$ cc -g -o buffer2 buffer2.c
```

```
$ ./buffer2
```

```
Enter value for name: Bill and Lawrie
```

```
Hello your name is Bill and Lawrie
```

```
buffer2 done
```

```
$ ./buffer2
```

```
Enter value for name: XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

```
Segmentation fault (core dumped)
```

36 byte (vediamo se sono sufficienti)

KO!

the Input data extend beyond the end of the buffer overwriting the saved frame pointer and return address.

Then, when the function attempts to transfer control to the return address, it typically jumps to an illegal memory location, resulting in a Segmentation Fault and the abnormal termination of the program,

OK

Memory Address	Before gets(inp)	After gets(inp)	Contains Value of
....			
bffffbe0	3e850408 >...	00850408	tag
bffffbdc	f0830408	94830408	return addr
bffffbd8	e8fbffbf	e8ffffbf	old base ptr
bffffbd4	60840408 '...	65666768 e f g h	
bffffbd0	30561540 0 V . @	61626364 a b c d	
bffffbcc	1b840408	55565758 U V W X	inp[12-15]
bffffbc8	e8fbffbf	51525354 Q R S T	inp[8-11]
bffffbc4	3cfcffbf <...	45464748 E F G H	inp[4-7]
bffffbc0	34fcffbf 4...	41424344 A B C D	inp[0-3]
....			

- **Vogliamo invocare 2 volte la funzione**
- Forniamo come input una **stringa** che permetta di arrivare fino alla locazione dell' Old frame pointer
- Forniamo poi un indirizzo per old frame pointer (puo' essere Old frame pointer o altro indirizzo valido e.g. **0xbffffe8**)
- Forniamo poi l'indirizzo della locazione di memoria a cui si trova il codice della funzione (assumiamo che sia **08048394**)
- Codifica indirizzi Little Endian (viene memorizzato a partire dal byte meno significativo)

```
$ perl -e 'print pack("H*",
  "414243444546474851525354555657586162636465666768
e8ffffbf948304080a4e4e4e4e0a");' | ./buffer2
```

Enter value for name:

Hello your Re?pyy]uEA is ABCDEFGHQRSTUVWXabcdefghijklm

Enter value for **Kyyu**:

Hello your **Kyyu** is **NNNN**

Segmentation fault (core dumped)

second time run: the tag parameter was referenced relative to the **0xbffffe8** overwritten saved frame pointer value, which is some location in upper memory, hence the garbage string Kyyu

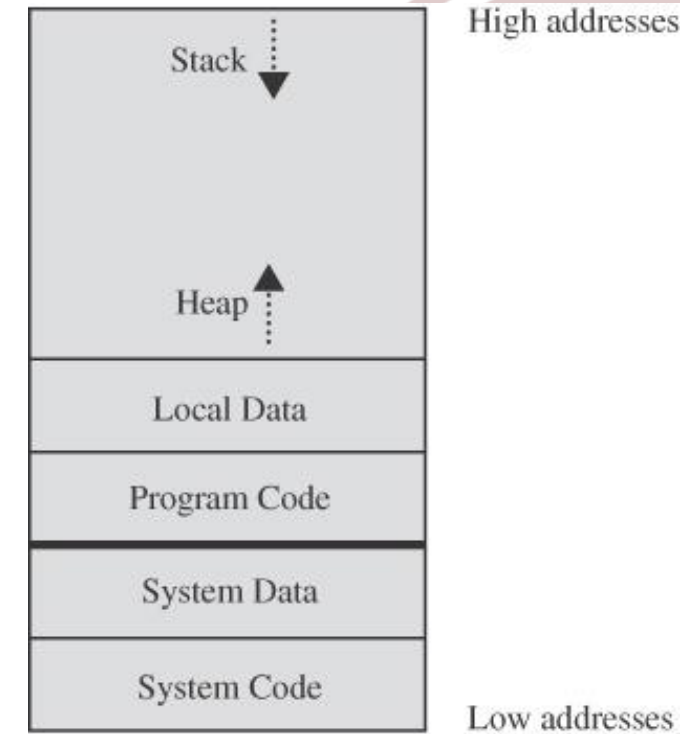


Codifica little-endian
 Es: 0x08048394
 →
 94 83 04 08

Memory Address	Before gets(inp)	After gets(inp)	Contains Value of
....			
bffffbe0	3e850408 >...	00850408	tag
bffffbdc	f0830408	94830408	return addr
bffffbd8	e8fbffbf	e8ffffbf	old base ptr
bffffbd4	60840408 '...	65666768 e f g h	
bffffbd0	30561540 0 V . @	61626364 a b c d	
bffffbcc	1b840408	55565758 U V W X	inp[12-15]
bffffbc8	e8fbffbf	51525354 Q R S T	inp[8-11]
bffffbc4	3cfcffbf <...	45464748 E F G H	inp[4-7]
bffffbc0	34fcffbf 4...	41424344 A B C D	inp[0-3]
....			

Overwriting the stack from the heap: Stack smashing

- In the heap are stored variable for which the size can change dynamically or is not defined at compile time, i.e. buffers for which they can grow according to the input data
- The heap can smash into the stack, although the OS has monitors their sizes and prevents such a collision, an unpredictable input can result in the heap overwhelming the stack
- **Reading:** *Aleph One (Elias Levy). "Smashing the Stack for Fun and Profit." Phrack, v7, n49, Nov 1996.*
 - <https://phrack.org/issues/49/14>



Summarizing

- An attacker can produce the following effects from an overflow attack:
 - **Overwrite the program counter** stored in the stack so that when this routine exits, control transfers to the address pointed at by the modified program counter address.
 - **Overwrite part of the code** in low memory, substituting the attacker's instructions for previous program statements.
 - **Overwrite the program counter and data** in the stack so that the program counter now points into the stack, causing the data overwritten into the stack to be executed.
- If the overflow occurs during a system call, that is, when the system is running with elevated privileges, the attacker's code also executes with those privileges

Overflow Countermeasures



To stay within bounds

Responsibility shared among the programmer, operating system, compiler, and hardware.

All should do the following:

- Check lengths before writing.
- Confirm that array subscripts are within limits.
- Double-check boundary condition code to catch possible off-by-one errors.
- Monitor input and accept only as many characters as can be handled.
- Use string utilities that transfer only a bounded amount of data.
- Check procedures that might overrun their space.
- Limit programs' privileges, so if a piece of code is overtaken maliciously, the violator does not acquire elevated system privileges as part of the compromise



Programming control

Techniques that are used to catch overflow situations before they become problems e.g.

- **code reviews**, in which people other than the programmer inspect code for implementation oversights
- **independent testing**, in which dedicated testers hypothesize points at which a program could fail



Programming language features

Some programming languages have features that preclude overflows, e.g.,

- Java, .NET, Perl, and Python generate code to check bounds before storing data
- the unchecked languages C, C++, and assembler language allow largely unlimited program access
 - To counter the openness of these languages, compiler writers have developed extensions and libraries that generate code to keep programs in check



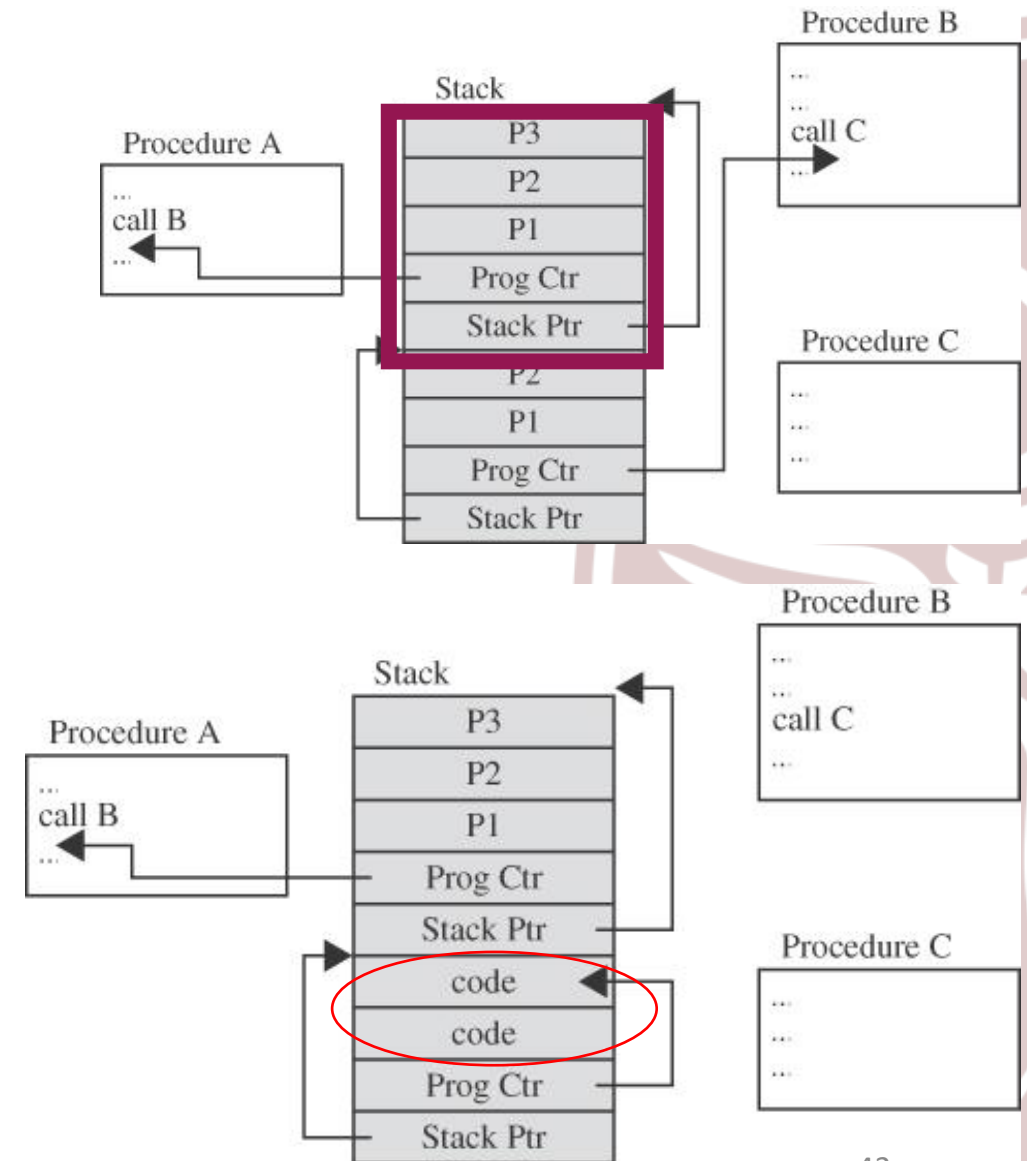
Code analyzer

- **Static code analyzer**, tools to analyze source code to detect unsafe conditions.
- The National Institute of Standards and Technology maintains a list of code analyzers at nist.gov/itl/ssd/software-quality-group/source-code-security-analyzers.



Canary (OS)

- The goal is to protect the stack
- The solution is wrapping each stack frame in a protective layer
- Assume the buffer overflow modify the stack overwriting the parameters of the subprocedure with code to execute and the program counter to point to this code



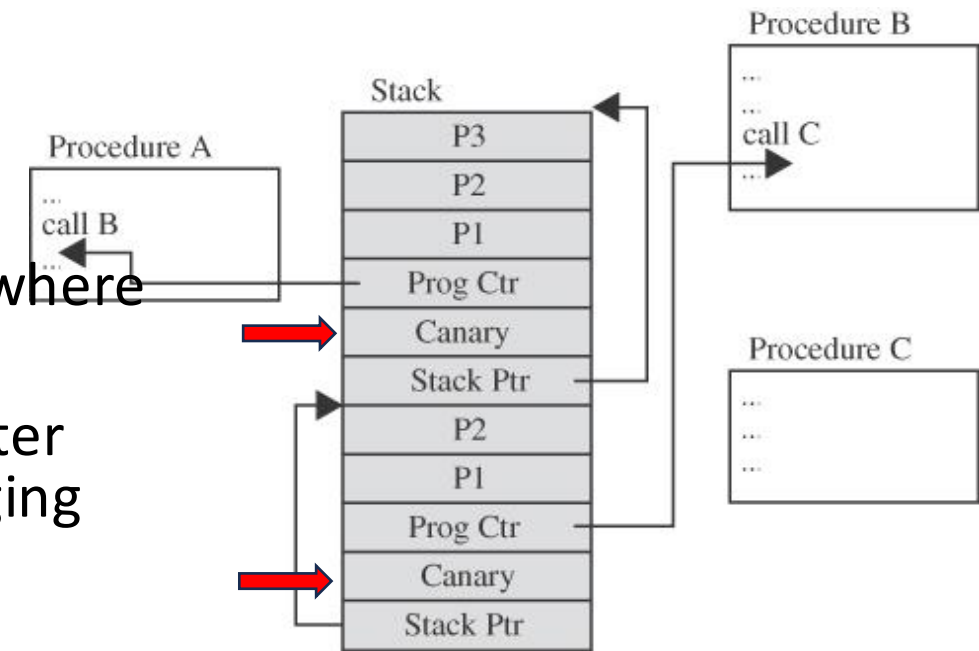
Canary (Cont'd)

Attack

- The attacker only know the approximate address where the program counter is in the stack
- The attacker has to rewrite not just the stack pointer but also some words around it to be sure of changing the true stack pointer

Defense

- Each procedure includes
 - a **prolog code** that create the stack frame
 - a **termination code** that cleans up the stack, reloads registers, and returns.
- The canary is as a tamper-detecting value: if the attacker try to modify the program counter also the canary is modified and the **termination code can be instrumented to check a canary value change**



It works as long as only the defender can generate and verify that value!!!

Incomplete mediation



Mediation

- Mediation means checking: the process of intervening to confirm an actor's authorization before a program takes an intended action.
- Is a flaw in access control
 - Verifying that the subject is authorized to perform the operation on an object is called **mediation**
 - **Incomplete mediation** – incomplete because a flaw!!!



An Incomplete mediation example

Consider the url

`http://www.somesite.com/subpage/userinput.asp?parm1=(808)555-1212&parm2=2019Jan17`

parm1 and parm2 pass data to the `userinput.asp` program

- What if parm1 and parm2 are different from what the program expect?
 - **The input should be carefully validated**
 - The programmer could do that with a well constructed web form
- However, **the application is still vulnerable.**
 - The programmer left these data fields in a place where the attacker can access (and modify) them.

The data values are not **completely mediated**: The sensitive data are in an exposed, uncontrolled condition.

A real example

- `http://www.things.com/order.asp?custID=101&part=555A&qy=20&price=10&ship=boat&shipcost=5&total=205`

`Total=qy*price+shipcost`

attack

What an attacker can do?

A real example

- `http://www.things.com/order.asp?custID=101&part=555A&qy=20&price=10&ship=boat&shipcost=5&total=205`

`Total=qy*price+shipcost`

Flaw:

`Total` computed at the ecommerce site fronted mediation is incomplete

- `http://www.things.com/order.asp?custID=101&part=555A&qy=20&price=1&ship=boat&shipcost=5&total=25`

attack



What could be the solution?



Countermeasure: Complete mediation

- Access control, implemented according to the reference monitor concept
- The three properties of a reference monitor are
 - (1) small and simple enough to give confidence of correctness,
 - (2) unbypassable, and
 - (3) always invoked.
- Properties (2) and (3) give us complete mediation
- **Input validation should be unbypassable and always invoked**



Time-of-Check to Time-of-Use

To improve efficiency, modern processors and OS usually change the order in which instructions and procedures are executed

- The **time-of-check to time-of-use** flaw concerns mediation that is performed with a “bait and switch” in the middle
- Between access check and use, data must be protected against change

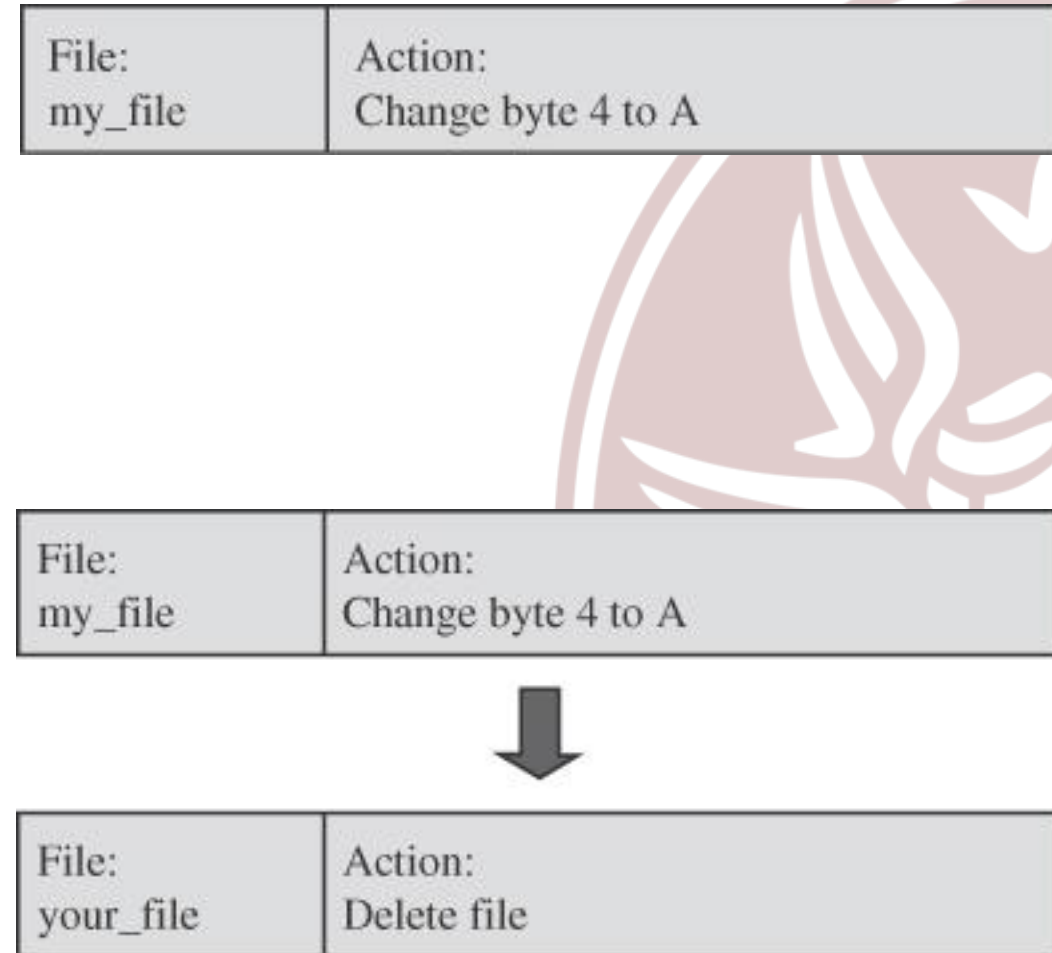
E.g. In a shop, at the counter I put 5x20\$ to pay a bill of 100\$ on the desk, the cashier got distracted by another customer and I take back 20\$, then the cashier takes the money and give me the bag and receipt.

Between the time the cashier checked the money and the time he/she complete the transaction something nasty appended!!!!

Another example

Suppose a request to access a file were presented as a data structure, including the name of the file and the mode of access

- the process implementing AC receives the data structure, determines whether the access should be denied or allows the access and forwards the data structure to the file handler for processing
 - **that could require the execution of many instructions**
 - **the incomplete mediation flaw can be exploited**



Security implications

- Time-of-check to time-of-use flaw exploits the delay between the two actions: check and use.
- Time-of-check to time-of-use flaw is an example of ineffective access control, leading to confidentiality failure, integrity failure, or both.



Countermeasures

- Verify the access-checking software own the request data until the requested action is complete
- Ensure serial integrity, that is, to allow no interruption (loss of control) during the validation
- Copy data from the user's space to the AC routine's area—out of the user's reach—and perform validation checks on the copy
- Seal the request data to detect modification

All these protection methods are expansions on the tamperproof criterion for a reference monitor:

Data on which the access control decision is based and the result of the decision must be outside the domain of the program whose access is being controlled.



Undocumented Access Point

- During program development and testing the programmer creates an undocumented entry point or execution mode (to facilitate the testing)
- These situations are understandable during program development
- **The flaw arise if the programmer forgets to remove these entry points when the program moves from development to product**

Backdoor or Tapdoor is the name for an undocumented access point. Such an entry can transfer control to any point with any privileges the programmer wanted

Protecting Against Unauthorized Entry

Rigorous code reviews in a software development process in theory, but in practice doesn't work

- being undocumented, these entry points will not be clearly labeled in source code or any of the development documentation
- such backdoors are often added after ordinary code development, during testing or even maintenance, so even the scrutiny of skilled reviewers will not find them
- maintenance people who add such code are seldom security engineers, so they are not used to thinking of vulnerabilities and failure modes



Other programming errors that can result in buffer overflow or other type of flaws

- Off-by-One Error
- Integer Overflow
- Unterminated Null-Terminated String
- Parameter Length, Type, and Number
 - Too many parameters
 - Wrong output type or size
 - Too-long string.
- Unsafe Utility Program

Max. len.	Curr. len.
20	5

(a) Separate length

HELLO

5HELLO

(b) Length precedes string

HELLOØ

(c) String ends with null

Race Condition (serialization flaw)



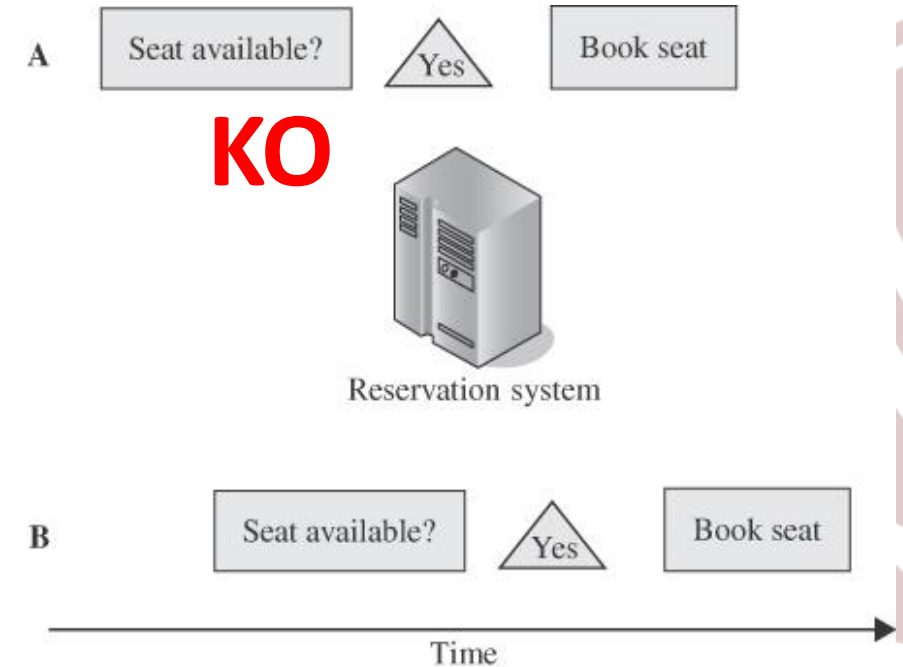
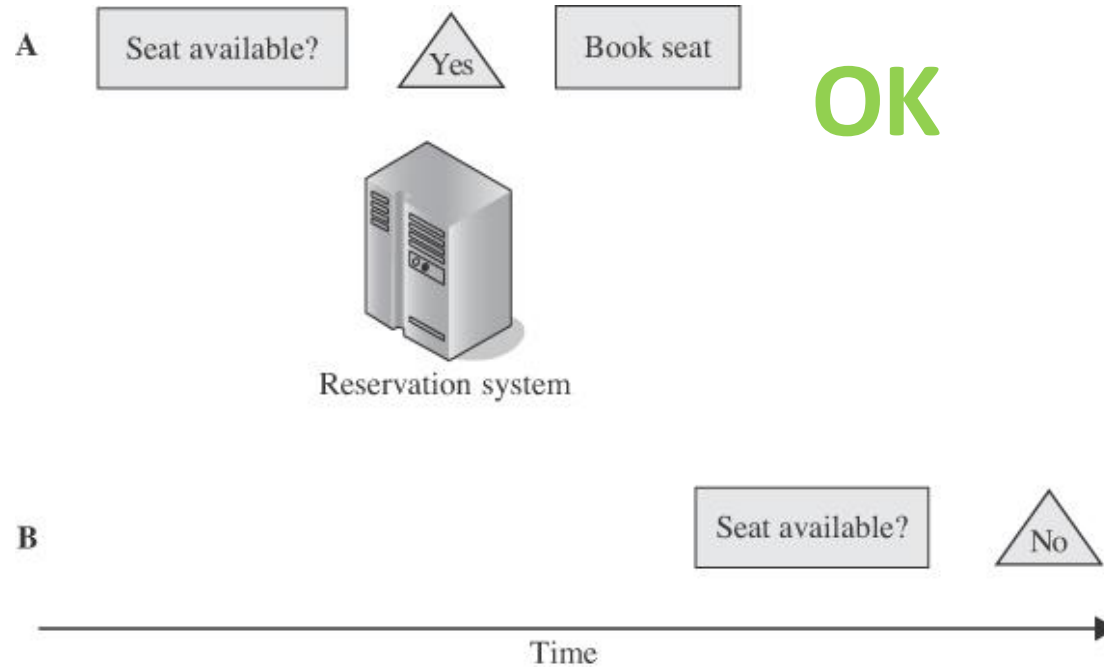
Race condition

Situation in which program behavior depends on the order in which two procedures execute

A race condition occurs most often in an operating system, but it can also occur in multithreaded or cooperating processes.



Unsynchronized Activity



- A race condition is difficult to detect
- it depends on the order in which two processes execute, that depends on
 - the total load on the system,
 - the amount of available memory space,
 - the priority of each process,
 - the number and timing of system interrupts to the processes.

Security implications

- A race condition between two processes can cause inconsistent, undesired, and therefore wrong outcomes—a **failure of integrity**.

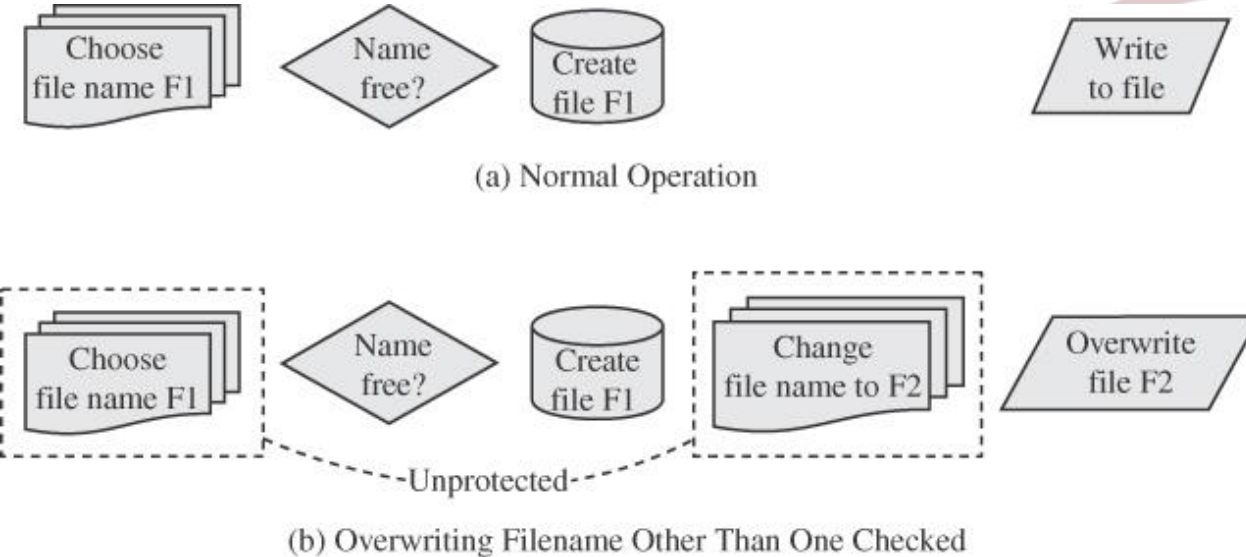


The Tripwire example

A utility for preserving the integrity of files

It creates a temporary file to which it writes a log of its activity. In the old version, Tripwire

- (1) chose a name for the temporary file
- (2) checked the file system to ensure that no file of that name already existed
- (3) created a file by that name, and (4) later opened the file and wrote results



- Difficult to synchronize for the attacker
- It could start from too early attempt and retry until success

Summary

Type of flaw	CIA property compromised
Incomplete mediation	Integrity, Availability
Buffer Overflow	All
Time-of-check to time-of-use	Confidentiality, Integrity
Backdoor	All
Race condition	Integrity